

PLATAFORMA DE CONOCIMIENTO CON IA

Cómo funciona el bot, hoy, de punta a punta

Arquitectura real del chatbot en producción: ingesta, chunking, embeddings, búsqueda híbrida, reranking, orquestación con LLM e intenciones que aprenden solas — y la decisión detrás de cada pieza.

GUÍA TÉCNICA PARA CHARLA · v1.0

Julio 2026 · Uso interno

Anclado al código real del repositorio — se distingue lo activo de lo aspiracional

Índice

1. Visión general: qué es, arquitectura en capas y stack real
2. El viaje de una consulta (el mapa completo)
3. Multitenancy: cómo aislamos a cada cliente
4. Ingesta de documentos

Reconocimiento de archivos · extracción · clasificación · chunking · quality gate · almacenamiento

5. Embeddings: qué son, qué modelo y por qué
6. Recuperación (retrieval): búsqueda híbrida + reranker
7. Pipeline de consulta: orquestador, cache, prompt y LLM
8. Intenciones dinámicas: el sistema que aprende solo
9. Canales: widget, WhatsApp y derivación a humano
10. Infraestructura: Docker, Redis, Celery, observabilidad
11. Resumen de decisiones de diseño (la lámina clave)
12. Estado real: qué está activo y qué está apagado hoy

1 · Visión general

Es una **plataforma SaaS multi-tenant** que permite a una organización consultar su conocimiento institucional en lenguaje natural. Sube documentos, el sistema los procesa con IA, y responde preguntas en ~1–3 segundos citando las fuentes, sin inventar. Cada cliente (tenant) ve solo sus propios datos, completamente aislados de los demás.

El corazón es un motor **RAG (Retrieval-Augmented Generation)**: en vez de que el LLM «recuerde», recuperamos los fragmentos relevantes del conocimiento del cliente y le pedimos al LLM que **redacte una respuesta usando solo eso**. Así evitamos alucinaciones y mantenemos la respuesta anclada a la fuente.

1.1 Arquitectura en capas

CLIENTE	→ Navegador web · Widget embebido · WhatsApp
FRONTEND	→ Next.js 14 + React 18 + TypeScript
GATEWAY	→ Nginx (única superficie pública: 80/443, SSL, rate limit)
BACKEND	→ FastAPI async + Celery workers (tareas en background)
IA / ML	→ LLM (Groq/OpenAI) + embeddings e5-large + reranker bge + HDBSCAN
DATOS	→ PostgreSQL · Qdrant · Redis · MinIO (Neo4j presente, hoy off)

1.2 Stack real (lo que corre hoy)

Rol	Tecnología	Nota
API backend	FastAPI (Python 3.11, async)	4 workers uvicorn detrás de Nginx
Frontend	Next.js 14 + React + TypeScript + Tailwind	Panel admin, dashboard, chat
Cola de tareas	Celery + Redis	Ingesta, clustering nocturno, reentrenamiento
Base relacional	PostgreSQL 16	Fuente de verdad: metadatos, usuarios, logs, parents
Búsqueda vectorial	Qdrant	Embeddings de chunks e intenciones (1 colección por tenant)
Cache + colas	Redis (3 DBs: broker · cache · rate-limit)	Respuestas, embeddings, sesiones, throttling
Archivos originales	MinIO (S3-compatible)	El documento tal cual lo subió el usuario
LLM principal	Groq: llama-3.3-70b-versatile (rápido) + llama-4-scout-17b (razonamiento)	En <i>local</i> se usa OpenAI gpt-4o-mini (el free-tier de Groq satura el tope diario)
Embeddings	multilingual-e5-large (1024 dims)	Multilingüe (español). Backends: local / TEI / OpenAI
Reranker	bge-reranker-large	En prod corre en un container TEI dedicado
Clustering	HDBSCAN + UMAP	Descubre intenciones nuevas de noche

Decisión — ¿por qué Groq? Es el proveedor de menor latencia del mercado para modelos Llama. La latencia es el atributo de calidad #1 del sistema (objetivo 2–4 s). No gestionamos modelos LLM propios para mantener la infra simple. (Los IDs de modelo se validan al arrancar: llama-3.1-405b y llama-4-maverick están prohibidos porque no existen en el tier de Groq y devuelven 404.)

2 · El viaje de una consulta

Este es el mapa que conviene tener en la cabeza toda la charla. Cada caja se detalla más adelante.

```
Usuario escribe: "¿tengo que sacar turno para el cardiólogo?"
|
1. Normalización      expande siglas (DNI, RRHH...), limpia espacios
|
2. ¿Cache?           exacto (Redis, hash) → semántico (Qdrant, coseno ≥0.97)
|   hit → responde en ~50 ms                miss ↓
3. Clasificar intención embedding → colección {tenant}_intenciones → label + confianza
|
4. Reescritura de query genera 1 variante para robustez (opcional)
|
5. RECUPERACIÓN (híbrida)
|   ├── Qdrant denso (top 100, por significado)
|   ├── expansión a "parent" (busca chico, responde con contexto grande)
|   ├── BM25 léxico en español (palabras exactas)
|   ├── fusión RRF (denso + léxico votan)
|   └── RERANKER cross-encoder (reordena los mejores 15)
|
6. Armado del prompt  system = personalidad + contexto + reglas anti-alucinación
|                    user  = SOLO la pregunta del usuario (aislada)
7. LLM redacta        Groq: modelo rápido (~80%) o de razonamiento (~20%)
|
8. Respuesta al usuario + evalúa si conviene derivar a un humano
|
9. Log async (Celery)  guarda la consulta → alimenta el aprendizaje de intenciones
```

Etapas 1–9 con timeouts independientes: si una etapa lenta falla (ej. reranker), degrada con gracia en vez de tirar toda la consulta.

3 · Multitenancy — cómo aislamos a cada cliente

Un solo despliegue sirve a muchas organizaciones. El aislamiento es **estructural**, no un simple `WHERE tenant_id=...` que se puede olvidar.

Store	Estrategia de aislamiento	Por qué
PostgreSQL	Schema separado por tenant: <code>tenant_<id>.documentos</code> , etc. El middleware setea <code>SET search_path</code> en cada conexión.	Un query sin filtro va al schema equivocado y falla, en vez de filtrar datos ajenos. El error es imposible a nivel de motor.
Qdrant	Colección separada: <code>{tenant}_docs</code> , <code>{tenant}_intenciones</code>	La búsqueda vectorial nunca cruza clientes.
Redis	Prefijo de clave: <code>{tenant}:cache:...</code>	Cache y sesiones namespaced por tenant.
MinIO	Prefijo de path: <code>{tenant}/{doc_id}/...</code>	Archivos originales segregados.

Decisión — schema-per-tenant y no columna `tenant_id` compartida. La columna compartida tiene riesgo de fuga si un query olvida el filtro. Un schema separado hace ese error **imposible a nivel del motor de base de datos**. El overhead es mínimo para el volumen esperado (<500 tenants). El `tenant_id` se resuelve del subdominio o del JWT (nunca de un header manipulable), se sanitiza contra injection, y tras el `SET search_path` se verifica con `SHOW` que realmente quedó aplicado — si no, corta ruidoso.

4 · Ingesta de documentos

Todo empieza cuando un admin sube un archivo. La respuesta HTTP es inmediata (**202 Accepted**); el trabajo pesado ocurre en background (Celery), así la subida nunca bloquea.

4.1 Reconocimiento de archivos

Aceptamos **PDF, DOCX, TXT, HTML, JSON**. El tipo se detecta en tres capas defensivas:

1. **MIME declarado** por el navegador. Si manda el genérico `application/octet-stream` (típico en `.docx`), se resuelve **por la extensión** del archivo.
2. **Verificación real con `libmagic`**: se leen los *magic bytes* reales del contenido. Si el tipo real no coincide con lo permitido → se rechaza (415) y se loguea como posible *spoofing*.
3. **Validaciones**: vacío → 400; >200 MB → 413; límite por plan (cantidad de docs y tamaño) → 402/413.

Decisión — validar el contenido real, no confiar en la extensión. Un atacante puede renombrar un ejecutable a `.pdf`. `libmagic` mira los bytes reales; el fallback por extensión solo cubre el caso legítimo del navegador que manda MIME genérico.

Deduplicación

Antes de procesar se calculan dos hashes SHA-256: de los **bytes crudos** y del **texto extraído normalizado**. Así detectamos duplicados aunque el archivo se re-exporte con metadatos distintos (mismo contenido, distintos bytes) → responde 409 con el tipo de coincidencia.

4.2 Extracción de texto

Formato	Librería	Detalle
PDF	<code>pypdf</code>	Texto por página, unido con dobles saltos de línea
DOCX	<code>python-docx</code>	Headings marcados como <code>##</code> ; tablas volcadas a <code>celda celda</code> ; opcionalmente convertidas a lenguaje natural
HTML	parser propio	Saltea <code>script / style</code>
JSON	aplanado recursivo	A líneas <code>clave: valor</code>
TXT	decode UTF-8	Con reemplazo tolerante

4.3 Clasificación del documento (sin LLM, en milisegundos)

Antes de trocear, un clasificador determinista mira la *forma* del texto (densidad de headers, artículos legales, listas, tablas, longitud de oraciones) y lo cataloga en **6 tipos**, cada uno con su estrategia de chunking:

Tipo	Ejemplo	Estrategia de troceo
<code>short</code>	Nota corta	Documento entero = 1 fragmento
<code>faq</code>	Preguntas frecuentes	Cada par pregunta/respuesta = 1 sección
<code>entity_list</code>	Listado de personas/áreas	Cada ítem <code>1.2 ...</code> = 1 sección
<code>structured</code>	Reglamento, contrato	Corte por artículos/capítulos/secciones
<code>mixed</code>	Mezcla	Fragmentos algo más chicos
<code>freeform</code>	Texto corrido	Corte estructural por párrafos

Decisión — clasificar la forma antes de trocear. No todos los documentos se cortan igual: partir un reglamento por artículos preserva la unidad legal; partir una FAQ por par pregunta-respuesta preserva la respuesta completa. Un troceo ciego mezclaría mitades de dos preguntas en un fragmento y arruinaría la recuperación.

4.4 Chunking jerárquico — «buscar chico, responder grande»

Esta es una de las decisiones más importantes del sistema. En vez de trocear en pedazos fijos, generamos **dos niveles**:

- **Parents** (secciones): unidades grandes con sentido completo (hasta ~700 palabras, ~2000 para reglamentos). Se guardan en PostgreSQL.
- **Children** (hijos): sub-fragmentos chicos de ~150 palabras (con 15 de solapamiento). Son los que se embeben y se buscan en Qdrant.

DOCUMENTO

- └ PARENT (sección, ~700 palabras) → guardado en PostgreSQL (tabla parent_chunks)
 - └ child 1 (~150 palabras) → embebido → Qdrant
 - └ child 2 (~150 palabras) → embebido → Qdrant (cada child apunta a su parent_id)
 - └ child 3 (~150 palabras) → embebido → Qdrant

En búsqueda: matcheás el CHILD (preciso) → pero al LLM le mandás el texto del PARENT (contexto)

Decisión — *small-to-big*. Un fragmento chico da un **match más preciso** (menos ruido en el vector), pero un fragmento chico como respuesta pierde contexto. Resolvemos las dos cosas: buscamos con el hijo chico y respondemos con el padre grande. Librería: RecursiveCharacterTextSplitter de LangChain, cortando por \n\n → \n → ". " → " " para no partir a mitad de oración.

Nota honesta para la charla: el CLAUDE.md describe un MVP de «fixed-size 512/64 plano» — eso quedó atrás. El código *hoy* ya hace chunking jerárquico parent-child con expansión small-to-big. Existe además un troceo *semántico* (agrupa oraciones por similitud) implementado pero **desconectado** del pipeline productivo.

4.5 Quality Gate — marca, no censura

Cada sección pasa por dos validaciones antes de indexarse:

- **Etapla 1 — autonomía (heurística, sin LLM):** descarta fragmentos demasiado cortos para tener sentido propio.
- **Etapla 2 — coherencia factual (LLM):** Groq juzga si el fragmento es coherente y auto-contenido. Devuelve {coherente, confianza, razón} .

Comportamiento clave: el quality gate **marca, no borra**. Un fragmento juzgado poco coherente se indexa igual con estado `pending` y participa en la búsqueda; queda visible para el admin en el panel. **Es mejor tener el chunk marcado que perder el documento.** Si Groq está caído, el chunk se marca `pending` y se re-valida más tarde con backoff (1m → 5m → 30m → 2h); solo el rechazo manual del admin lo saca de la búsqueda.

4.6 Dónde termina cada cosa

Store	Qué guarda
Qdrant	Un vector (1024-dim) por child , con payload: texto, <code>document_id</code> , <code>parent_id</code> , <code>quality_gate_status</code> , metadatos (filename, section_header, flags de contacto...)
PostgreSQL	<code>documentos</code> (metadatos, estado, chunk_count) · <code>parent_chunks</code> (texto completo de cada sección + índice <i>tsvector</i> para BM25)
MinIO	El archivo original , tal cual se subió
Neo4j	Desactivado hoy — la extracción de entidades a grafo está comentada (ver §12)

5 · Embeddings — qué son y por qué estos

Un **embedding** convierte un texto en un vector de números donde la **cercanía geométrica** $\approx$ **cercanía de significado**. «Sacar turno» y «pedir cita» caen cerca aunque no compartan palabras. Eso es lo que permite buscar *por significado* y no por coincidencia literal.

Modelo	intfloat/multilingual-e5-large
Dimensiones	1024
Idiomas	100+, incluye español (crítico: el corpus es en español)

Decisión — e5-large multilingüe, NUNCA bge-large-en. bge-large-en es **solo inglés**. Usar un modelo inglés sobre corpus español degrada la calidad semántica desde el primer documento. Está prohibido por el validador de config. e5-large tiene la misma dimensionalidad (1024), así que es compatible con Qdrant sin cambios.

El detalle que sorprende: los prefijos `query: / passage:`

e5 se entrenó **asimétrico**: a las *preguntas* hay que anteponerles `"query: "` y a los *textos del corpus* `"passage: "`. Sin el prefijo correcto los vectores caen en subespacios distintos y la similitud se degrada. El sistema los agrega automáticamente en cada lado.

Tres backends intercambiables

Backend	Cuándo	Trade-off
local	Corre dentro del proceso FastAPI	~1.5 GB RAM, usa CPU/GIL → puede frenar el event loop bajo carga
TEI	Container dedicado (HuggingFace TEI, Rust)	Mismo modelo, <i>batching</i> dinámico → ~10× throughput
OpenAI	text-embedding-3-small vía API	~80 ms, libera RAM del backend, sin GIL — se fuerza a 1024-dim para compatibilidad

local y TEI usan el mismo modelo (se puede cambiar sin re-embeddear). Cambiar a/desde OpenAI sí requiere re-embeddear todo (vectores de modelos distintos no son comparables).

Optimización aplicada: los embeddings de **preguntas se cachean en Redis 24 h** (key = hash del texto normalizado). Embeber cuesta 200–500 ms; preguntas repetidas o iguales entre usuarios reutilizan el vector. Además hay reintentos con backoff ante rate-limit (429): sin ellos, un throttle de OpenAI devolvía vacío y el bot decía «no encontré» aunque el dato existiera.

6 · Recuperación (retrieval) — búsqueda híbrida + reranker

Recuperar bien es el 80% de la calidad de un RAG. No usamos una sola técnica: combinamos **cuatro**, cada una tapando el punto ciego de la otra.

6.1 Búsqueda densa (Qdrant)

Se embebe la pregunta y se buscan los **top 100** chunks más cercanos por coseno. Es la búsqueda «por significado». Los chunks marcados `skipped` por el quality gate reciben una penalización ($\times 0.85$): participan pero se despriorizan.

6.2 Expansión a parent (small-to-big)

Los matches son *children* chicos. Antes de seguir, se traen de PostgreSQL los **textos de los parents** correspondientes y se reemplaza el texto del hijo por el del padre (dedup por parent, quedándose con el hijo de mayor score). Match preciso, contexto completo.

6.3 BM25 léxico (español) + RRF

En paralelo corre una búsqueda **léxica** clásica (BM25 sobre *tsvector* en español, en PostgreSQL). Captura coincidencias **exactas** de palabra que el embedding a veces difumina (números, nombres propios, siglas). Los dos rankings —denso y léxico— se fusionan con **RRF** (*Reciprocal Rank Fusion*): cada chunk suma $1/(60 + \text{su_posición})$ en cada lista. Los que aparecen bien rankeados en ambas suben.

Decisión — BM25 con AND, no OR. Con OR, una palabra común («horario») traía chunks genéricos que dominaban el ranking léxico y, vía RRF, enterraban los buenos resultados del embedding. Con AND, en preguntas multi-palabra matchea poco o nada → no contamina; y si da cero, cae limpio a solo-embedding.

Decisión — denso + léxico juntos. El embedding entiende sinónimos pero difumina lo exacto; BM25 clava lo exacto pero no entiende sinónimos. Juntos se cubren mutuamente. RRF los combina sin tener que calibrar escalas incompatibles.

6.4 El reranker — la autoridad final de relevancia

Acá está la pieza que más conviene explicar bien en la charla.

Bi-encoder (el embedding)

Codifica pregunta y documento **por separado**. Rápido (comparás vectores pre-calculados), pero pierde matices de la interacción pregunta ↔ documento.

Cross-encoder (el reranker)

Mete (**pregunta, chunk**) **juntos** en el mismo pase del modelo y produce un score de relevancia mucho más fino. Entiende que «cardiólogo» ≈ «Cardiología». Es caro → solo sobre los mejores candidatos.

Modelo: `bge-reranker-large` (en prod, container TEI). Reordena los candidatos ya filtrados y se queda con los **top 15**, normalizando el score a $[0,1]$ (sigmoid) para tener una escala de confianza usable.

Decisión — cuándo NO correr el reranker. Es caro (~2 s de CPU). Se **saltea inteligentemente** si: (a) está deshabilitado; (b) el tenant tiene menos de N documentos (nada que reordenar); (c) hay menos de 5 candidatos (Qdrant ya los ordenó bien). Rerankear 3 elementos no mejora nada y suma latencia. Tiene timeout: si tarda de más, cae a ordenar por el score previo.

6.5 Multi-query

Si la reescritura generó variantes de la pregunta, se recupera con cada una **en paralelo** y se fusiona con RRF: los chunks que aparecen en más variantes se priorizan. El reranker final puntúa el conjunto fusionado con la pregunta *original*.

La regla mental: «RRF = selección de candidatos; reranker = puntuación de relevancia». Uno junta a los sospechosos, el otro decide quién es realmente relevante.

7 · Pipeline de consulta — el orquestador

El orquestador coordina todo lo anterior y arma la respuesta. Pasos:

1. **Normalización:** expande siglas argentinas (DNI, CUIT, RRHH, AFIP...) y colapsa espacios, para alinear pregunta, embedding y cache.
2. **Cache en dos niveles:** primero *exacto* (hash de la pregunta en Redis → ~50 ms); si falla, *semántico* (busca en Qdrant una pregunta previa con coseno ≥ 0.97 y reusa su respuesta).
3. **Clasificación de intención** (§8) — en paralelo.
4. **Reescritura de query** (opcional): genera 1 variante para robustez ante «vocabulary mismatch».
5. **Recuperación híbrida** (§6).
6. **Filtro de relevancia y armado de contexto:** ordena y recorta hasta ~15 chunks. Si el mejor score es muy bajo, marca «baja confianza».
7. **Armado del prompt** (abajo).
8. **Llamada al LLM** con routing de modelo.
9. **Cache + log async** (Celery, no bloquea la respuesta).

7.1 El prompt: separación system / user (anti prompt-injection)

```
system:  personalidad del bot
        + "=== SOBRE ESTA ORGANIZACIÓN ==="    (descripción del tenant)
        + "=== ALCANCE TEMÁTICO ==="            (qué puede/no puede responder)
        + contexto recuperado (los chunks)
        + reglas anti-alucinación
user:    <SOLO la pregunta del usuario, sanitizada, aislada>
```

Decisión — el input del usuario JAMÁS se concatena al prompt. Va siempre en una variable aislada, en el turno `user`, nunca interpolado en las instrucciones (`f"...{user_input}..."` está prohibido). Aunque el usuario escriba «ignora tus instrucciones», sus palabras están en un compartimento separado de las reglas. Además se sanitiza (se quitan caracteres de control, se trunca).

7.2 Routing de modelo LLM

Complejidad	Modelo	Cuándo
Simple (~80%)	llama-3.3-70b-versatile	Consulta estándar. ~1.2 s
Compleja (~20%)	llama-4-scout-17b	>20 palabras, o varias preguntas juntas. ~3 s

La elección es una heurística barata (conteo de palabras y signos de pregunta), no otra llamada al LLM. En local el proveedor es OpenAI `gpt-4o-mini`.

7.3 Anti-alucinación

Tres capas: **(1)** el LLM recibe la instrucción explícita de responder *solo* con el contexto provisto y admitir cuando no sabe; **(2)** si no hay chunks suficientemente buenos, se responde un mensaje determinista de «no tengo esa información» *sin* llamar al LLM; **(3)** las respuestas de baja confianza se marcan. Medición reciente en un cliente real: *faithfulness* 0.89, *refusal* correcto 1.00, alucinación 0.04.

8 · Intenciones dinámicas — el sistema que aprende solo

Las intenciones (temas recurrentes: «sacar turno», «horarios», «recetas») **no se definen a mano**: emergen de las consultas reales. Este es el mecanismo que hace que el bot mejore solo con el uso.

8.1 Reconocimiento en runtime

El clasificador es una colección Qdrant (`{tenant}_intenciones`) con ejemplos de cada intención. Clasificar = embeber la pregunta y ver a qué ejemplo se parece más. Según el coseno, tres bandas:

Confianza	Acción
alta ($\geq 0.95^*$)	Clasifica con certeza; candidata a auto-aprendizaje
media (0.70–0.94*)	Responde igual; encola para revisión del admin
baja / desconocida	Responde con RAG; la consulta queda para el clustering nocturno

*Umbral por defecto 0.95 / 0.70. En clientes reales se calibraron más bajo (0.85 / 0.55) porque con e5 casi nada llega a 0.95 y quedaba mucha cobertura afuera. Son configurables por env.

8.2 El ciclo completo (cómo nace una intención)

1. CONSULTA NUEVA
| el bot no la reconoce (banda baja) → responde RAG
| y guarda la consulta en `consultas_log` (unassigned)
2. CLUSTERING 02:00
| HDBSCAN agrupa las consultas sin intención
| (UMAP baja de 1024–15 dims primero, si no HDBSCAN ve todo ruido)
| grupos con ≥ 15 consultas → "candidatos"
3. COHERENCIA (Groq)
| ¿el grupo es un tema coherente? Groq lo valida
| └ sí → se crea la INTENCIÓN (label + ejemplos → Qdrant)
| └ incoherente → se SUBDIVIDE en sub-temas finos y se reintentan
4. CONSOLIDACIÓN
| fusiona intenciones redundantes (por centroide + nombre)
5. REENTRENAMIENTO 03:00
| re-indexa ejemplos; mide precisión; si baja → ROLLBACK
6. YA SE RECONOCE
| la próxima consulta igual cae en banda media/alta
7. AUTO-APRENDIZAJE
| si llega con confianza ≥ 0.95 , se agrega como ejemplo
| (tope: máx 30% de ejemplos auto-aprendidos por intención)

8.3 Las salvaguardas (y por qué)

UMAP antes de HDBSCAN. En 1024 dimensiones, HDBSCAN clasifica casi todo como ruido (maldición de la dimensionalidad). UMAP reduce a ~15 dims preservando la estructura, y ahí HDBSCAN sí encuentra grupos.

Gate de coherencia con Groq. HDBSCAN a veces sobre-fusiona sub-temas distintos. Antes de mostrarle un grupo al admin (o auto-promoverlo), Groq valida que sea un tema coherente. Si no lo es, en vez de descartarlo se **subdivide** con un clustering más fino — así no perdemos cobertura de sub-temas reales (recetas, estudios, órdenes...).

Reentrenamiento con rollback automático. Cada reentrenamiento mide la precisión sobre consultas ya clasificadas (ground truth real). Si la versión nueva baja más de 5% respecto a la anterior, **revierte sola** borrando los vectores nuevos. El clasificador nunca empeora en silencio.

Cap del 30% de auto-aprendizaje. Si dejáramos que el bot aprenda de sí mismo sin límite, un sesgo se amplificaría (drift). Nunca más del 30% de los ejemplos de una intención son auto-aprendidos; pasado el tope, los nuevos van al panel del admin como «pendientes», no se descartan.

Por qué HDBSCAN y no K-Means: K-Means exige decir de antemano cuántos grupos hay. HDBSCAN los **descubre** de los datos — fundamental, porque no tenemos intenciones predefinidas.

9 · Canales y derivación a humano

- **Widget web embebible:** JS que se pega en cualquier sitio del cliente. Autentica con un `widget_token` de solo-lectura (revocable), sin exponer credenciales. Mantiene conversación con sesión.
- **WhatsApp:** integración por tenant vía WhatsApp Cloud API (Meta). La cuenta se configura desde el panel; los secretos se guardan **cifrados** (Fernet). El webhook entrante se rutea al tenant por el `phone_number_id` y valida la firma HMAC.
- **Derivación a operador humano (handoff):** tras responder, un evaluador decide si conviene ofrecer un humano (según calidad de la respuesta y señales de frustración). Si el usuario acepta, la conversación pasa a un operador en el panel, con historial inmutable de quién atendió.

Decisión — el widget no usa cookies ni el JWT de usuario. Se embebe en sitios de terceros (cross-origin). Un token dedicado de scope limitado (solo lectura, solo queries, solo ese tenant, 90 días, revocable por hash) evita exponer sesiones reales y se puede revocar sin afectar al resto del tenant.

10 · Infraestructura

Pieza	Detalle
Orquestación	Docker Compose. Única superficie pública: Nginx (80/443). Las bases solo son accesibles dentro de la red Docker (bind a loopback del host).
Redis — 3 DBs	DB0 broker de Celery (jobs) · DB1 cache de respuestas y embeddings (efímero) · DB2 rate limiting. Separadas a propósito: si Redis se reinicia, no se mezcla la pérdida de jobs con la de cache.
Celery — cron nocturno	01:30 limpieza · 02:00 clustering · 02:30 auto-promoción · 03:00 reentrenamiento . Colas separadas: ingest, clustering, training.
Rate limiting	Por tenant (600/min) y por IP en el widget (30/min), en Redis DB2. Fail-open si Redis cae.
Observabilidad	Prometheus + Grafana + Loki (métricas, logs); tracing por spans en el pipeline.
Backups	PostgreSQL con pgBackRest + WAL → S3 (diario + point-in-time). Qdrant se regenera desde los documentos originales de MinIO.

11 · Resumen de decisiones de diseño

La lámina para cerrar la charla: cada decisión y su porqué en una línea.

Decisión	Por qué
RAG en vez de LLM «a secas»	Anclar la respuesta a la fuente del cliente → sin alucinaciones, actualizable subiendo docs
Groq como LLM	La menor latencia del mercado para Llama; la latencia es el atributo #1
Schema-per-tenant	Aislamiento imposible de romper por un query olvidado
e5-large multilingüe (no bge-en)	El corpus es español; un modelo inglés degrada todo
Prefijos query/passage	e5 se entrenó asimétrico; sin prefijo, cae la similitud
Clasificar el documento antes de trocear	Cada tipo se corta distinto; preservar la unidad de sentido
Chunking jerárquico (small-to-big)	Buscar chico = preciso; responder grande = con contexto
Quality gate que marca, no borra	Mejor un chunk marcado que perder el documento
Búsqueda híbrida denso + BM25 + RRF	Semántica y léxica se cubren los puntos ciegos
Reranker cross-encoder	Juicio de relevancia fino que el coseno no captura
Saltear el reranker cuando no aporta	No pagar ~2 s de latencia si hay pocos candidatos
Cache de respuestas y de embeddings	~50 ms en hits; ahorrar 200–500 ms de embedding
Input del usuario aislado en el prompt	Anti prompt-injection estructural
Intenciones que emergen (HDBSCAN)	No hay que definir las a mano; el bot mejora con el uso
Rollback automático del clasificador	Nunca empeora en silencio
Cap 30% de auto-aprendizaje	Evita el <i>drift</i> por aprender de sí mismo

12 · Estado real — activo vs. apagado hoy

Honestidad técnica para la charla: qué está corriendo y qué es diseño presente-pero-desactivado. Conviene saberlo para no prometer de más.

Componente	Estado	Nota
RAG (embed → híbrido → rerank → LLM)	ACTIVO	El camino principal, en producción
Chunking jerárquico parent-child	ACTIVO	Ya no es el «fixed 512/64» del CLAUDE.md
Búsqueda híbrida + reranker (TEI)	ACTIVO	En prod; en local el reranker está apagado
Intenciones dinámicas + clustering nocturno	ACTIVO	Con auto-promoción y reentrenamiento
Widget + WhatsApp + handoff	ACTIVO	—
Neo4j + extracción de entidades (GLiNER)	APAGADO	Código presente pero comentado (<code>ENTITIES_DISABLED</code>). El grafo Persona → Rol → Horario no se puebla hoy
Chunking semántico	APAGADO	Implementado pero desconectado del pipeline productivo
Corte duro anti-alucinación por score	APAGADO	El umbral absoluto (<code>min_score</code>) está en 0 porque la escala del score no es estable; el anti-alucinación vive en el prompt. Se reactiva cuando el reranker corra siempre

Lo que esto significa: el sistema es un **RAG híbrido maduro con intenciones auto-evolutivas**. Las piezas de grafo (Neo4j/GLiNER) son la evolución planificada, no lo que responde hoy. La charla debería centrarse en lo que está activo — que ya es bastante sofisticado.